

Introduction to Java

ULTIMATE JAVA

Complete Study Notes

BCA / B.Tech — Full Syllabus

Part 1: Introduction → OOP Concepts

Exam-Ready • Concise • Code-First

Topics: Intro to Java | Data Types | Operators | Control Statements | Arrays | Strings | OOP

UNIT 1 — Introduction to Java

1.1 History of Java

Java was developed by James Gosling at Sun Microsystems in 1991 under the project name 'Oak'. It was officially released in 1995 and renamed 'Java'. Later, Sun was acquired by Oracle Corporation in 2010.

Key Points (Exam Ready)

- Created by: James Gosling (Sun Microsystems)
- Original name: Oak → renamed to Java in 1995
- Purpose: Originally designed for set-top boxes and embedded systems
- Version: Java 1.0 released in 1995; now Java 21+ (LTS)
- Currently maintained by: Oracle Corporation

► 1.1.1 Features of Java (WORA Principle)

Java follows the principle: Write Once, Run Anywhere (WORA). This means Java code compiled on one platform can run on any other platform without recompilation.

15 Features of Java

- Simple — Easy syntax, no pointers, no memory management
- Object-Oriented — Everything is an object (except primitives)
- Platform Independent — .class bytecode runs on any OS via JVM
- Secure — No explicit pointers; bytecode verification by JVM
- Robust — Strong type checking, exception handling, garbage collection
- Multithreaded — Built-in support for concurrent programming
- Architecture Neutral — Compiler generates architecture-neutral bytecode
- Portable — Consistent behavior across platforms
- High Performance — JIT (Just-In-Time) compiler speeds up execution
- Distributed — Supports networking via java.net, RMI
- Dynamic — Supports dynamic loading of classes at runtime
- Interpreted — Bytecode is interpreted by JVM
- Compiled — Source code compiled to bytecode (.class)
- Garbage Collected — Automatic memory management
- Strongly Typed — Every variable must have a declared type

Golden Rules / Important Notes

- Most asked feature: Platform Independence (WORA)
- Pointers are NOT supported in Java — this makes it secure & simple
- Java is both compiled AND interpreted (trick question!)

Exam-Oriented Questions

- Q: What is Java? List any 5 features of Java. (Theory)
- Q: What does WORA mean in Java? Explain with diagram.
- Q: Write a note on the history and features of Java.

1.2 JVM, JDK, and JRE

These three are the core components of the Java platform. Understanding them is essential for every exam.

► JVM — Java Virtual Machine

JVM is a virtual machine that provides a runtime environment to execute Java bytecode. It is platform-specific (different JVM for Windows, Linux, Mac).

```
Java Source (.java)
  ↓ javac (compiler)
Bytecode (.class)
  ↓ JVM (platform-specific)
Machine Code (execution)
```

JVM Responsibilities

- Load, verify, and execute bytecode
- Memory management (Heap, Stack, Method Area)
- Garbage Collection
- Security — Bytecode Verifier checks malicious code
- JIT Compilation — converts bytecode to native code at runtime

► JRE — Java Runtime Environment

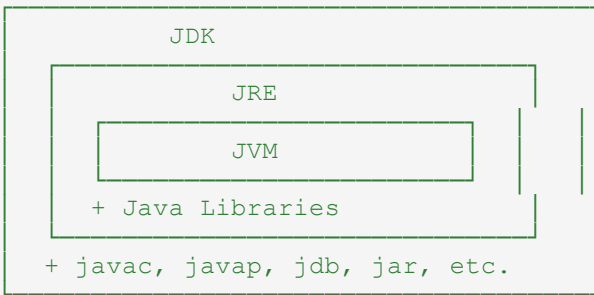
- JRE = JVM + Library Classes (java.lang, java.util, etc.)
- Used by end-users to RUN Java programs
- Does NOT include development tools (no javac compiler)

► JDK — Java Development Kit

- JDK = JRE + Development Tools (javac, java, javadoc, jar, etc.)
- Used by DEVELOPERS to write, compile, and run Java programs
- Includes: javac (compiler), java (interpreter), javap, jdb (debugger)



Remember: $\text{JDK} \supseteq \text{JRE} \supseteq \text{JVM}$



⚡ Golden Rules / Important Notes

- JDK for developers, JRE for users, JVM for running bytecode
- Platform independence is achieved because bytecode is same, JVM is different
- JIT compiler is inside JVM — improves performance at runtime

📝 Exam-Oriented Questions

- Q: Differentiate between JVM, JRE, and JDK. (6 marks)
- Q: Explain the role of JVM in achieving platform independence.

1.3 First Java Program

Every Java program starts with a class. The main() method is the entry point.



Syntax:

```
// File name must match class name: Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

```
}  
}
```



Output:

```
Hello, World!
```

► Line-by-Line Explanation

- — public: accessible everywhere; class: blueprint; Hello: class name
- — Entry point. JVM calls this to start the program
- — Can be called without creating object
- — main() does not return any value
- — Command-line arguments (array of strings)
- — Prints to console with newline



Compilation & Execution:

```
javac Hello.java    // Compiles → generates Hello.class  
java Hello          // Runs the bytecode
```



Golden Rules / Important Notes

- File name MUST match public class name (case-sensitive)
- main() signature must be exactly: public static void main(String[] args)
- System.out.print() → no newline; System.out.println() → with newline



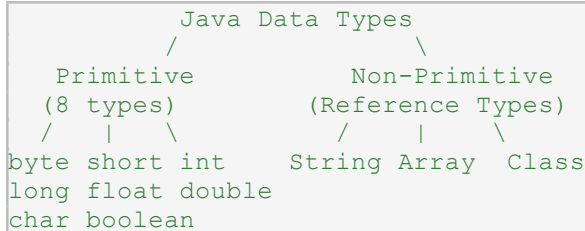
Exam-Oriented Questions

- Q: Write a Java program to print 'Hello World'. Explain each keyword.
- Q: What happens if file name doesn't match class name?

UNIT 2 — Data Types, Variables & Operators

2.1 Data Types in Java

Data types define what kind of value a variable can store. Java has two categories of data types.



► 2.1.1 Primitive Data Types (8 types)

Type	Size	Default	Range	Example
byte	1 byte	0	-128 to 127	byte b = 100;
short	2 bytes	0	-32768 to 32767	short s = 5000;
int	4 bytes	0	-2 ³¹ to 2 ³¹ -1	int i = 100000;
long	8 bytes	0L	-2 ⁶³ to 2 ⁶³ -1	long l = 99L;
float	4 bytes	0.0f	7 decimal digits	float f = 9.8f;
double	8 bytes	0.0d	15 decimal digits	double d = 3.14;
char	2 bytes	'\u0000'	0 to 65535 (Unicode)	char c = 'A';
boolean	1 bit	false	true / false	boolean b = true;

► 2.1.2 Non-Primitive / Reference Data Types

- — Sequence of characters: String name = "Java";
- — Collection of same type: int[] arr = {1,2,3};
- — User-defined type
- — Contract for classes

Golden Rules / Important Notes

- char in Java is 2 bytes (Unicode) — NOT 1 byte like in C/C++
- String is NOT a primitive type — it is a class (reference type)
- long literals end with L: 99L; float with f: 9.8f
- boolean only accepts true/false — NOT 0/1 like in C

Exam-Oriented Questions

- Q: List and explain all primitive data types in Java with size and range.
- Q: What is the difference between primitive and non-primitive data types?

2.2 Variables in Java

A variable is a named memory location that stores a value. In Java, every variable must be declared with a data type.

Syntax:

```
data_type variable_name = value;
int age = 20;
String name = "Rahul";
double pi = 3.14;
```

► Types of Variables

- — Declared inside method/block; no default value; must initialize before use
- — Declared inside class but outside methods; has default value; one per object
- — Declared with 'static' keyword; shared by all objects; one per class

```
public class VarDemo {
    int x = 10;           // Instance variable
    static int count = 0; // Static variable

    void show() {
        int local = 5;    // Local variable
        System.out.println(x + " " + count + " " + local);
    }
}
```

► Type Casting

- — Smaller → Larger: int to double (automatic)
- — Larger → Smaller: double to int (manual, data loss possible)

```
int a = 10;
double d = a;           // Widening - automatic
double x = 9.99;
int b = (int) x;         // Narrowing - explicit cast → b = 9 (truncated)
```

Golden Rules / Important Notes

- Local variables have NO default value — must initialize before use
- Instance variables get default values (int→0, boolean→false, String→null)
- Narrowing cast can cause data loss!

2.3 Operators in Java

Operators are symbols that perform operations on variables and values.

► 2.3.1 Arithmetic Operators

```
int a = 10, b = 3;
System.out.println(a + b); // 13 (Addition)
System.out.println(a - b); // 7  (Subtraction)
System.out.println(a * b); // 30 (Multiplication)
System.out.println(a / b); // 3  (Integer Division)
System.out.println(a % b); // 1  (Modulus/Remainder)
```

w► 2.3.2 Relational / Comparison Operators

- == (equal to), != (not equal), > (greater than), < (less than)
- >= (greater or equal), <= (less or equal)
- Returns: true or false

► 2.3.3 Logical Operators

- — true if BOTH conditions are true

- — true if AT LEAST ONE condition is true
- — reverses the boolean value

► 2.3.4 Assignment Operators

```
int x = 10;
x += 5;    // x = x + 5 = 15
x -= 3;    // x = x - 3 = 12
x *= 2;    // x = x * 2 = 24
x /= 4;    // x = x / 4 = 6
x %= 4;    // x = x % 4 = 2
```

► 2.3.5 Increment / Decrement Operators

```
int a = 5;
System.out.println(a++); // 5 (post-increment: use then increase)
System.out.println(a);   // 6
System.out.println(++a); // 7 (pre-increment: increase then use)
System.out.println(a--); // 7 (post-decrement)
System.out.println(--a);  // 5 (pre-decrement)
```

► 2.3.6 Bitwise Operators

- & (Bitwise AND), | (Bitwise OR), ^ (XOR), ~ (Complement)
- << (Left shift), >> (Right shift), >>> (Unsigned right shift)

► 2.3.7 Ternary Operator

Shortest form of if-else. Syntax: condition ? value_if_true : value_if_false

```
int a = 10, b = 20;
int max = (a > b) ? a : b; // max = 20
System.out.println("Max = " + max);
```



Output:

```
Max = 20
```

► 2.3.8 Operator Precedence (High to Low)

```
() → ++/-- → * / % → + -
→ << >> → < > <= >=
→ == != → & → ^ → |
→ && → || → ?: → = += -=
```

⚡ Golden Rules / Important Notes

- a++ vs ++a: Both increment but a++ returns OLD value, ++a returns NEW value
- / operator on integers gives integer result: 10/3 = 3, NOT 3.33
- % gives remainder: 10%3 = 1
- Ternary operator is great for simple if-else in exams — use it!



Exam-Oriented Questions

- Q: Explain different types of operators in Java with examples.
- Q: What is the difference between ++a and a++? Explain with program.
- Q: Write Java code to find maximum of two numbers using ternary operator.

UNIT 3 — Control Statements

3.1 Decision Making Statements

► 3.1.1 if Statement

```
if (condition) {  
    // executes if condition is true  
}
```

```
int marks = 75;  
if (marks >= 40) {  
    System.out.println("Pass");  
}
```

 **Output:**

```
Pass
```

► 3.1.2 if-else Statement

```
int marks = 30;  
if (marks >= 40) {  
    System.out.println("Pass");  
} else {  
    System.out.println("Fail");  
}
```

 **Output:**

```
Fail
```

► 3.1.3 if-else-if Ladder

```
int marks = 82;  
if (marks >= 90)      System.out.println("Grade A+");  
else if (marks >= 80) System.out.println("Grade A");  
else if (marks >= 70) System.out.println("Grade B");  
else if (marks >= 40) System.out.println("Grade C");  
else                  System.out.println("Fail");
```

 **Output:**

```
Grade A
```

► 3.1.4 switch Statement

switch is used when one variable needs to be compared to multiple fixed values. More efficient than if-else-if for many fixed comparisons.

```
int day = 3;  
switch (day) {  
    case 1: System.out.println("Monday");   break;  
    case 2: System.out.println("Tuesday");  break;  
    case 3: System.out.println("Wednesday"); break;  
    default: System.out.println("Other");  
}
```

 **Output:**

```
Wednesday
```

Golden Rules / Important Notes

- break in switch is MANDATORY — without it, fall-through happens (executes ALL cases below)
- switch works with: int, char, byte, short, String (Java 7+), enum

- switch does NOT work with: float, double, long
- default is like 'else' — executes when no case matches

3.2 Looping Statements

► 3.2.1 for Loop

Use when the number of iterations is KNOWN.

```
// Syntax
for (initialization; condition; update) {
    // body
}

// Example: Print 1 to 5
for (int i = 1; i <= 5; i++) {
    System.out.print(i + " ");
}
```



Output:

```
1 2 3 4 5
```

► 3.2.2 while Loop

Use when the number of iterations is NOT known. Checks condition BEFORE executing body.

```
int i = 1;
while (i <= 5) {
    System.out.print(i + " ");
    i++;
}
```



Output:

```
1 2 3 4 5
```

► 3.2.3 do-while Loop

Executes the body AT LEAST ONCE — checks condition AFTER execution.

```
int i = 1;
do {
    System.out.print(i + " ");
    i++;
} while (i <= 5);
```



Output:

```
1 2 3 4 5
```

Loop Comparison:

for	while	do-while
Known iters	Unknown iters	Runs at least 1
Entry ctrl	Entry ctrl	Exit controlled
Compact	Flexible	Menu-driven app

► 3.2.4 Enhanced for Loop (for-each)

Used to iterate over arrays and collections. Simple syntax, no index needed.

```
int[] nums = {10, 20, 30, 40, 50};
for (int n : nums) {
```

```
    System.out.print(n + " ");  
}
```

Output:

```
10 20 30 40 50
```

► 3.2.5 break and continue

```
// break - exits the loop immediately  
for (int i = 1; i <= 10; i++) {  
    if (i == 5) break;  
    System.out.print(i + " ");  
}  
// Output: 1 2 3 4  
  
// continue - skips current iteration  
for (int i = 1; i <= 5; i++) {  
    if (i == 3) continue;  
    System.out.print(i + " ");  
}  
// Output: 1 2 4 5
```

Golden Rules / Important Notes

- do-while always executes at least once — even if condition is false initially
- Infinite loop: while(true) { } — use break to exit
- break exits loop; continue skips current iteration but loop continues
- Enhanced for (for-each) cannot modify array elements directly

Exam-Oriented Questions

- Q: Differentiate between while and do-while loop with example.
 - Q: Write a Java program to print multiplication table of any number using for loop.
 - Q: Write a program to find factorial of a number using while loop.
-

UNIT 4 — Arrays

4.1 Introduction to Arrays

An array is a collection of elements of the SAME data type stored in contiguous memory locations. Arrays have fixed size once declared.

Key Points

- Arrays are objects in Java (stored in heap memory)
- Index starts from 0
- Size is fixed after initialization
- Default values: int→0, double→0.0, boolean→false, String→null
- `ArrayIndexOutOfBoundsException` if accessing invalid index

4.2 1D Arrays

Declaration & Initialization:

```
// Method 1: Declare then initialize
int[] arr = new int[5];           // {0,0,0,0,0} by default
arr[0] = 10; arr[1] = 20;         // Assign values

// Method 2: Declare with values
int[] marks = {85, 92, 78, 90, 88};

// Method 3: Anonymous array
System.out.println(new int[]{1,2,3}[0]); // 1
```

Program: 1D Array — Input, Sum, Average

```
import java.util.Scanner;
public class ArrayDemo {
    public static void main(String[] args) {
        int[] marks = {85, 90, 78, 92, 88};
        int sum = 0;
        // Traverse array
        for (int i = 0; i < marks.length; i++) {
            sum += marks[i];
        }
        double avg = (double) sum / marks.length;
        System.out.println("Sum = " + sum);
        System.out.println("Average = " + avg);
    }
}
```

Output:

```
Sum = 433
Average = 86.6
```

4.3 2D Arrays

A 2D array is like a table (matrix) with rows and columns.

```
// Declaration
int[][] matrix = new int[3][3]; // 3x3 matrix

// Initialization with values
int[][] mat = {
    {1, 2, 3},
```

```

        {4, 5, 6},
        {7, 8, 9}
    };

    // Access element: mat[row][col]
    System.out.println(mat[1][2]); // 6 (row 1, col 2)

```

Program: Matrix Display

```

public class Matrix {
    public static void main(String[] args) {
        int[][] mat = {{1,2,3},{4,5,6},{7,8,9}};
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(mat[i][j] + "\t");
            }
            System.out.println(); // new row
        }
    }
}

```

Output:

```

123
456
789

```

4.4 Array Methods (java.util.Arrays)

```

import java.util.Arrays;

int[] arr = {5, 2, 8, 1, 9};
Arrays.sort(arr); // Sort: {1,2,5,8,9}
System.out.println(Arrays.toString(arr)); // [1, 2, 5, 8, 9]
int idx = Arrays.binarySearch(arr, 5); // Returns index of 5
int[] copy = Arrays.copyOf(arr, 3); // {1, 2, 5}

```

⚡ Golden Rules / Important Notes

- `arr.length` gives size — no parentheses (not a method, it's a property)
- `Arrays.sort()` sorts in ASCENDING order by default
- 2D array: rows = `mat.length`, cols = `mat[0].length`
- Passing array to method passes REFERENCE (changes affect original)

📝 Exam-Oriented Questions

- Q: What is an array? Explain 1D and 2D arrays with syntax and examples.
- Q: Write a Java program to find the largest element in an array.
- Q: Write a program to add two matrices using 2D arrays.

UNIT 5 — Strings in Java

5.1 Introduction to Strings

A String in Java is an object of the String class (not a primitive). Strings are immutable — once created, they cannot be changed.

Key Points

- String is a class in java.lang package
- Strings are immutable (value cannot be changed after creation)
- String literals are stored in String Constant Pool (SCP)
- Two ways to create: String literal and using new keyword

```
String s1 = "Hello";           // String literal (in SCP)
String s2 = new String("Hello"); // New object (in Heap)
System.out.println(s1.equals(s2)); // true (content comparison)
System.out.println(s1 == s2);    // false (reference comparison)
```

5.2 Important String Methods

Method	Description	Example
<code>length()</code>	Returns length of string	"Hello".length() → 5
<code>charAt(i)</code>	Returns char at index i	"Hello".charAt(1) → 'e'
<code>substring(i, j)</code>	Returns substring from i to j-1	"Hello".substring(1,3) → "el"
<code>toUpperCase()</code>	Converts to UPPERCASE	"hi".toUpperCase() → "HI"
<code>toLowerCase()</code>	Converts to lowercase	"HI".toLowerCase() → "hi"
<code>trim()</code>	Removes leading/trailing spaces	" hi ".trim() → "hi"
<code>equals()</code>	Case-sensitive comparison	"a".equals("a") → true
<code>equalsIgnoreCase()</code>	Case-insensitive comparison	"A".equalsIgnoreCase("a") → true
<code>contains(s)</code>	Checks if string contains s	"Hello".contains("ell") → true
<code>indexOf(s)</code>	Returns index of first occurrence	"Hello".indexOf('l') → 2
<code>replace(a, b)</code>	Replaces a with b	"cat".replace('c', 'b') → "bat"
<code>split(regex)</code>	Splits by regex pattern	"a,b,c".split(",") → [a,b,c]
<code>startsWith(s)</code>	Checks if starts with s	"Hello".startsWith("He") → true
<code>endsWith(s)</code>	Checks if ends with s	"Hello".endsWith("lo") → true
<code>isEmpty()</code>	Checks if length is 0	"".isEmpty() → true
<code>concat(s)</code>	Joins two strings	"Hi".concat(" World") → "Hi World"

5.3 StringBuffer and StringBuilder

Unlike String, StringBuffer and StringBuilder are MUTABLE — their content can be changed without creating new objects.

Feature	String	StringBuffer	StringBuilder
Mutable?	No	Yes	Yes

Thread Safe?	Yes	Yes (sync)	No (faster)
Performance	Slow	Medium	Fast
Use when	Constant	Multi-thread	Single-thread

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");    // Hello World
sb.insert(5, ",");      // Hello, World
sb.delete(5, 6);        // Hello World
sb.reverse();           // dlroW olleH
System.out.println(sb.toString());
```

Golden Rules / Important Notes

- ALWAYS use .equals() to compare Strings, NOT == (== checks reference, not content)
- String is immutable: s = s + "!" creates a NEW String object each time
- StringBuilder is faster than StringBuffer (no synchronization overhead)
- String concatenation using + in loops is very slow — use StringBuilder

Exam-Oriented Questions

- Q: What is the difference between String, StringBuffer, and StringBuilder?
 - Q: Write a Java program to reverse a string without using built-in method.
 - Q: Explain any 5 methods of the String class with examples.
-

UNIT 6 — Object-Oriented Programming (OOP)

6.1 OOP Concepts Overview

Java is an Object-Oriented Language. OOP organizes code around objects and classes rather than functions and logic.

4 Pillars of OOP:

- | | |
|------------------|---------------------|
| 1. Encapsulation | – Data hiding |
| 2. Inheritance | – Reusability |
| 3. Polymorphism | – Many forms |
| 4. Abstraction | – Hiding complexity |

6.2 Class and Object

A class is a blueprint/template. An object is a real-world instance of that class.

```
// Class Definition
class Student {
    // Attributes (Instance Variables)
    String name;
    int age;
    int rollNo;

    // Method
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Roll No: " + rollNo);
    }
}

// Main class to create objects
public class Main {
    public static void main(String[] args) {
        // Creating objects
        Student s1 = new Student(); // Object creation
        s1.name = "Rahul";
        s1.age = 20;
        s1.rollNo = 101;
        s1.display();

        Student s2 = new Student(); // Another object
        s2.name = "Priya";
        s2.age = 19;
        s2.rollNo = 102;
        s2.display();
    }
}
```



Output:

```
Name: Rahul
Age: 20
Roll No: 101
Name: Priya
Age: 19
Roll No: 102
```

⚡ Golden Rules / Important Notes

- new keyword allocates memory in heap and calls constructor

- Each object has its OWN copy of instance variables
- Methods are shared among all objects of a class

6.3 Constructors

A constructor is a special method that is called automatically when an object is created. It initializes the object.



Constructor Rules

- Same name as class
- No return type (not even void)
- Called automatically with 'new' keyword
- If no constructor defined, Java provides default constructor
- Can be overloaded (multiple constructors with different parameters)

► Types of Constructors

```
class Rectangle {
    int length, width;

    // 1. Default Constructor (no parameters)
    Rectangle() {
        length = 1;
        width = 1;
    }

    // 2. Parameterized Constructor
    Rectangle(int l, int w) {
        length = l;
        width = w;
    }

    // 3. Copy Constructor
    Rectangle(Rectangle r) {
        length = r.length;
        width = r.width;
    }

    int area() { return length * width; }

    public static void main(String[] args) {
        Rectangle r1 = new Rectangle();           // Default
        Rectangle r2 = new Rectangle(5, 3);       // Parameterized
        Rectangle r3 = new Rectangle(r2);         // Copy
        System.out.println("r1 area: " + r1.area()); // 1
        System.out.println("r2 area: " + r2.area()); // 15
        System.out.println("r3 area: " + r3.area()); // 15
    }
}
```



Output:

```
r1 area: 1
r2 area: 15
r3 area: 15
```

► this Keyword

- 'this' refers to current object's reference
- Resolves conflict between instance variable and parameter names
- Can be used to call another constructor: this()

```

class Box {
    int height;
    Box(int height) {
        this.height = height; // 'this.height' = instance var
                             // 'height' = parameter
    }
}

```

Golden Rules / Important Notes

- Constructor is NOT inherited — subclass must define its own
- Constructors can be overloaded — same name, different parameters
- Default constructor is auto-provided only if NO constructor is defined
- 'this' keyword is used to avoid naming conflicts

Exam-Oriented Questions

- Q: What is a constructor? Explain types of constructors with examples.
- Q: What is constructor overloading? Write a program to demonstrate it.
- Q: Explain the 'this' keyword in Java with example.

6.4 Encapsulation

Encapsulation = Wrapping data (variables) and methods together in a class AND hiding the internal data using access modifiers. Achieved using private variables + public getters/setters.

```

class BankAccount {
    private double balance; // Hidden from outside (private)
    private String owner;

    // Constructor
    BankAccount(String owner, double initialBalance) {
        this.owner = owner;
        this.balance = initialBalance;
    }

    // Getter (Read access)
    public double getBalance() {
        return balance;
    }

    // Setter (Controlled Write access)
    public void deposit(double amount) {
        if (amount > 0) balance += amount; // Validation!
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) balance -= amount;
        else System.out.println("Insufficient funds");
    }

    public static void main(String[] args) {
        BankAccount acc = new BankAccount("Rahul", 1000);
        acc.deposit(500);
        acc.withdraw(200);
        System.out.println("Balance: " + acc.getBalance()); // 1300.0
    }
}

```

Golden Rules / Important Notes

- Encapsulation = private data + public methods (getters/setters)
- Advantage: Validation can be added in setters to protect data integrity
- A class with all private fields and public getters/setters is called a POJO (Plain Old Java Object) or JavaBean



Exam-Oriented Questions

- Q: What is encapsulation? How is it implemented in Java?
- Q: Write a program demonstrating encapsulation using a Student class.

6.5 Inheritance

Inheritance allows a child class (subclass) to acquire the properties and methods of a parent class (superclass). Main benefit: Code Reusability.

```
Types of Inheritance in Java:
Single:      A → B
Multilevel:  A → B → C
Hierarchical: A → B, A → C
Multiple:    NOT supported with classes (use interfaces)
Hybrid:      Mix of above (via interfaces)
```

► Single Inheritance

```
// Parent class
class Animal {
    String name;
    void eat() {
        System.out.println(name + " is eating");
    }
    void sleep() {
        System.out.println(name + " is sleeping");
    }
}

// Child class inherits from Animal
class Dog extends Animal {
    void bark() {
        System.out.println(name + " is barking!");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.name = "Buddy";
        d.eat();    // Inherited from Animal
        d.sleep();  // Inherited from Animal
        d.bark();   // Dog's own method
    }
}
```



Output:

```
Buddy is eating
Buddy is sleeping
Buddy is barking!
```

► super Keyword

- 'super' refers to the parent class object
- super.method() — calls parent's method
- super() — calls parent's constructor (must be first statement)

```
class Vehicle {
    Vehicle() { System.out.println("Vehicle created"); }
```

```

        void info() { System.out.println("I am a vehicle"); }
    }
    class Car extends Vehicle {
        Car() {
            super(); // Calls Vehicle() constructor
            System.out.println("Car created");
        }
        void info() {
            super.info(); // Calls Vehicle's info()
            System.out.println("I am a car");
        }
    }
}

```

► Multilevel Inheritance

```

class A { void methodA() { System.out.println("A"); } }
class B extends A { void methodB() { System.out.println("B"); } }
class C extends B {
    public static void main(String[] args) {
        C obj = new C();
        obj.methodA(); // Inherited from A
        obj.methodB(); // Inherited from B
    }
}

```

⚡ Golden Rules / Important Notes

- Java does NOT support multiple inheritance with classes (use interfaces)
- extends keyword is used for inheritance
- Constructor is NOT inherited, but super() calls parent's constructor
- Private members are NOT inherited

📝 Exam-Oriented Questions

- Q: What is inheritance? Explain types of inheritance with diagrams.
- Q: What is the use of super keyword? Explain with example.
- Q: Write a program showing multilevel inheritance.

6.6 Polymorphism

Polymorphism means 'many forms'. In Java, the same method name can behave differently in different contexts. Two types: Compile-time and Runtime.

► 6.6.1 Method Overloading (Compile-time Polymorphism)

Same method name, different parameters (number/type/order). Resolved at compile time.

```

class Calculator {
    // Overloaded methods - same name, different params
    int add(int a, int b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
    double add(double a, double b) { return a + b; }

    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(10, 20)); // 30
        System.out.println(c.add(10, 20, 30)); // 60
        System.out.println(c.add(1.5, 2.5)); // 4.0
    }
}

```

► 6.6.2 Method Overriding (Runtime Polymorphism)

Child class provides its OWN implementation of a method already defined in parent class. Resolved at runtime.

```

class Shape {
    void draw() {
        System.out.println("Drawing a shape");
    }
}

class Circle extends Shape {
    @Override // Optional but good practice
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s;
        s = new Circle(); // Runtime: Circle's draw()
        s.draw();
        s = new Rectangle(); // Runtime: Rectangle's draw()
        s.draw();
    }
}

```

Output:

```

Drawing a Circle
Drawing a Rectangle

```

Feature	Overloading	Overriding
Polymorphism type	Compile-time	Runtime
Class	Same class	Different classes
Parameters	Must differ	Must be same
Return type	Can differ	Must be same (or covariant)
Inheritance needed?	No	Yes

Golden Rules / Important Notes

- Overloading: SAME class, DIFFERENT params. Overriding: DIFFERENT class, SAME params
- @Override annotation is optional but helps catch errors at compile time
- Private and static methods CANNOT be overridden
- final method CANNOT be overridden

Exam-Oriented Questions

- Q: What is polymorphism? Differentiate between method overloading and overriding.
- Q: Write a program demonstrating runtime polymorphism using inheritance.

6.7 Abstraction

Abstraction = Hiding the implementation details and showing only the essential features. Achieved using abstract classes and interfaces.

► Abstract Class

Rules for Abstract Class

- Declared with 'abstract' keyword
- Can have both abstract (no body) and concrete (with body) methods
- Cannot be instantiated (cannot create objects directly)
- Subclass must implement all abstract methods (or also be abstract)

```
abstract class Shape {
    // Abstract method – no body (must be overridden)
    abstract double area();

    // Concrete method – has body (can be used directly)
    void describe() {
        System.out.println("I am a shape");
    }
}

class Circle extends Shape {
    double radius;
    Circle(double r) { radius = r; }

    @Override
    double area() { return Math.PI * radius * radius; }
}

class Square extends Shape {
    double side;
    Square(double s) { side = s; }

    @Override
    double area() { return side * side; }
}

public class Main {
    public static void main(String[] args) {
        Shape c = new Circle(7);
        Shape sq = new Square(5);
        c.describe();
        System.out.printf("Circle Area: %.2f\n", c.area());
        System.out.printf("Square Area: %.2f\n", sq.area());
    }
}
```



Output:

```
I am a shape
Circle Area: 153.94
Square Area: 25.00
```

Golden Rules / Important Notes

- If a class has even ONE abstract method, the class MUST be abstract
- Abstract class CAN have constructors (called via super())
- You CANNOT create objects of abstract class: new Shape() → ERROR
- Abstract class provides partial abstraction; Interface provides full abstraction



Exam-Oriented Questions

- Q: What is abstraction? How is it achieved in Java?
- Q: Write a program using abstract class to calculate area of Circle and Rectangle.
- Q: Can an abstract class have a constructor? Explain.

Advance Java

ADVANCE JAVA

Complete Study Notes

BCA / B.Tech — Full Syllabus

Part 2: Interfaces → JDBC

Interfaces | Packages | Exception Handling | Multithreading
File Handling | Collections | GUI (AWT/Swing) | JDBC

UNIT 7 — Interfaces and Packages

7.1 Interface

An interface is a 100% abstract blueprint. It defines WHAT a class should do, not HOW. Interfaces provide full abstraction and support multiple inheritance in Java.

Key Rules for Interface

- All methods are public and abstract by default (before Java 8)
- All variables are public, static, and final by default
- A class implements an interface using 'implements' keyword
- A class can implement MULTIPLE interfaces (solves multiple inheritance problem)
- Java 8+: interfaces can have default and static methods
- Java 9+: interfaces can have private methods

Syntax:

```
interface InterfaceName {
    // Abstract method (public abstract by default)
    void method1();
    int method2();

    // Default method (Java 8+)
    default void greet() {
        System.out.println("Hello from interface!");
    }
}

class MyClass implements InterfaceName {
    public void method1() { System.out.println("method1 implemented"); }
    public int method2() { return 42; }
}
```

► Multiple Interface Implementation

```
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

// Duck implements BOTH interfaces
class Duck implements Flyable, Swimmable {
    public void fly() { System.out.println("Duck is flying!"); }
    public void swim() { System.out.println("Duck is swimming!"); }
}

public class Main {
    public static void main(String[] args) {
        Duck d = new Duck();
        d.fly();
        d.swim();
    }
}
```

Output:

```
Duck is flying!
Duck is swimming!
```

► Interface vs Abstract Class

Feature	Abstract Class	Interface
Keyword	abstract class	interface
Methods	abstract + concrete	abstract (default in Java 8+)
Variables	any type	public static final only
Constructor	Yes	No
Multiple Inheritance	No	Yes
Use when	Partial abstraction	Full abstraction / contract

⚡ Golden Rules / Important Notes

- Interface provides multiple inheritance — class can implement many interfaces
- Interface variables are constants: public static final
- Cannot create object of interface: new Flyable() → ERROR
- If a class doesn't implement all interface methods → class must be abstract

📝 Exam-Oriented Questions

- Q: What is an interface in Java? How is it different from abstract class?
- Q: Write a program showing multiple interface implementation.
- Q: Can an interface extend another interface? How?

7.2 Access Modifiers

Access modifiers control the visibility (scope) of classes, methods, and variables.

Modifier	Same Class	Same Package	Subclass	Everywhere
private	✓ Yes	✗ No	✗ No	✗ No
default	✓ Yes	✓ Yes	✗ No	✗ No
protected	✓ Yes	✓ Yes	✓ Yes	✗ No
public	✓ Yes	✓ Yes	✓ Yes	✓ Yes

7.3 Packages

A package is a namespace/folder that groups related classes and interfaces. It avoids naming conflicts and provides access control.

📌 Benefits of Packages

- Avoids naming conflicts (two classes with same name in different packages)
- Provides access protection (default access = package-level)
- Makes code organized and maintainable

► Built-in Packages

- — Automatically imported. Contains String, Math, System, Object, etc.
- — Collections, Scanner, Arrays, Date, etc.
- — File, FileReader, BufferedReader, etc.

- — URL, Socket, ServerSocket for networking
- — Connection, Statement, ResultSet for JDBC
- — JFrame, JButton, JLabel for GUI

► Creating User-Defined Package

```
// File: mypackage/Hello.java
package mypackage; // Must be first statement

public class Hello {
    public void greet() {
        System.out.println("Hello from mypackage!");
    }
}

// File: Main.java (in different package)
import mypackage.Hello; // Import the class

public class Main {
    public static void main(String[] args) {
        Hello h = new Hello();
        h.greet();
    }
}
```

Compile & Run:

```
javac mypackage/Hello.java
javac Main.java
java Main
```



Golden Rules / Important Notes

- package statement must be the FIRST statement in Java file
- import java.lang.* is automatically done — no need to import explicitly
- import packagename.* imports all classes from a package
- Use fully qualified name: java.util.Scanner sc = new java.util.Scanner(System.in);



Exam-Oriented Questions

- Q: What is a package in Java? Explain with example.
- Q: What is the difference between import and package?
- Q: List any 5 built-in Java packages and their uses.

UNIT 8 — Exception Handling

8.1 What is an Exception?

An exception is an unexpected event that occurs during program execution and disrupts the normal flow. Exception handling allows the program to handle errors gracefully instead of crashing.

Key Terms

- **Exception:** An abnormal condition during runtime (e.g., divide by zero)
- **Error:** Serious problem (JVM crash, OutOfMemory) — cannot be handled
- **Checked Exception:** Checked at compile time (IOException, SQLException)
- **Unchecked Exception:** Checked at runtime (ArithmeticException, NullPointerException)

Exception Hierarchy:

```
Throwable
├── Error (cannot handle)
│   ├── OutOfMemoryError
│   └── StackOverflowError
└── Exception (can handle)
    ├── Checked (compile time)
    │   ├── IOException
    │   ├── SQLException
    │   └── ClassNotFoundException
    └── Unchecked (runtime - RuntimeException)
        ├── ArithmeticException
        ├── NullPointerException
        ├── ArrayIndexOutOfBoundsException
        ├── NumberFormatException
        └── ClassCastException
```

8.2 try-catch-finally

The try block contains code that might throw an exception. catch handles the exception. finally always executes (cleanup code).

Syntax:

```
try {
    // Risky code that might throw exception
} catch (ExceptionType e) {
    // Handle the exception
} finally {
    // Always executes (cleanup)
}
```

Example:

```
public class ExceptionDemo {
    public static void main(String[] args) {
        try {
            int a = 10, b = 0;
            int result = a / b; // ArithmeticException thrown here
            System.out.println(result); // This won't execute
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            System.out.println("Finally block always runs!");
        }
        System.out.println("Program continues...");
    }
}
```

```
}
```

Output:

```
Error: / by zero  
Finally block always runs!  
Program continues...
```

8.3 Multiple catch Blocks

```
public class MultiCatch {  
    public static void main(String[] args) {  
        try {  
            int[] arr = new int[5];  
            arr[10] = 100; // ArrayIndexOutOfBoundsException  
        } catch (ArithmeticException e) {  
            System.out.println("Arithmetic error: " + e);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array index error: " + e.getMessage());  
        } catch (Exception e) {  
            System.out.println("General error: " + e.getMessage());  
        }  
    }  
}
```

Output:

```
Array index error: Index 10 out of bounds for length 5
```

8.4 throw and throws

Difference

- throw: Used to manually throw an exception inside a method
- throws: Used in method signature to declare checked exceptions

```
// throw - manual exception  
class AgeValidator {  
    void checkAge(int age) {  
        if (age < 18) {  
            throw new IllegalArgumentException("Age must be 18+");  
        }  
        System.out.println("Age is valid: " + age);  
    }  
}  
  
// throws - declare checked exception  
import java.io.*;  
class FileReader {  
    void readFile(String name) throws IOException {  
        // May throw IOException  
        FileInputStream f = new FileInputStream(name);  
    }  
}
```

8.5 Custom / User-Defined Exception

```
// Create custom exception by extending Exception  
class InsufficientFundsException extends Exception {  
    InsufficientFundsException(String message) {  
        super(message); // Pass message to Exception  
    }  
}
```

```

class Account {
    double balance = 1000;

    void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException(
                "Cannot withdraw " + amount + ". Balance: " + balance
            );
        }
        balance -= amount;
        System.out.println("Withdrawn: " + amount + ". Remaining: " + balance);
    }

    public static void main(String[] args) {
        Account acc = new Account();
        try {
            acc.withdraw(500);
            acc.withdraw(800); // This will fail
        } catch (InsufficientFundsException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}

```



Output:

```

Withdrawn: 500.0. Remaining: 500.0
Exception: Cannot withdraw 800.0. Balance: 500.0

```



Golden Rules / Important Notes

- finally block always executes — even if exception is thrown or return is used
- Catch most specific exceptions first, then general (Exception) last
- throw vs throws: throw is an action; throws is a declaration
- Custom exception: extend Exception (checked) or RuntimeException (unchecked)
- NullPointerException is most common runtime exception in Java



Exam-Oriented Questions

- Q: What is exception handling? Explain try-catch-finally with example.
- Q: Differentiate between throw and throws.
- Q: Write a program to create a custom exception 'InvalidAgeException'.
- Q: What is the difference between checked and unchecked exceptions?

UNIT 9 — Multithreading

9.1 Introduction

Multithreading is the ability of a program to execute multiple threads simultaneously. Each thread is an independent unit of execution within a process.

Key Concepts

- Thread: Lightweight subprocess — smallest unit of processing
- Process: A running program (has its own memory)
- Multitasking: Multiple processes running (process-based)
- Multithreading: Multiple threads within one process (thread-based)
- Benefits: Better CPU utilization, responsive UI, faster execution

9.2 Thread Lifecycle

```
Thread States:
NEW → RUNNABLE → RUNNING → (BLOCKED/WAITING) → TERMINATED

NEW:          Thread created but start() not called
RUNNABLE:     start() called, waiting for CPU time
RUNNING:      Thread is executing (CPU is processing it)
BLOCKED:      Waiting for monitor lock
WAITING:      Waiting indefinitely (wait())
TIMED_WAIT:   Waiting for specific time (sleep())
TERMINATED:   Thread completed or stopped
```

9.3 Creating Threads — Two Ways

► Method 1: Extending Thread class

```
class MyThread extends Thread {
    private String threadName;

    MyThread(String name) { threadName = name; }

    @Override
    public void run() { // Code to execute in thread
        for (int i = 1; i <= 3; i++) {
            System.out.println(threadName + " - Count: " + i);
            try { Thread.sleep(500); } // Pause 500ms
            catch (InterruptedException e) { e.printStackTrace(); }
        }
    }
}

public class ThreadDemo {
    public static void main(String[] args) {
        MyThread t1 = new MyThread("Thread-1");
        MyThread t2 = new MyThread("Thread-2");
        t1.start(); // Starts the thread (calls run())
        t2.start();
    }
}
```

Output (order may vary — concurrent execution):

```
Thread-1 - Count: 1
```

```
Thread-2 - Count: 1
Thread-1 - Count: 2
Thread-2 - Count: 2
...
```

► Method 2: Implementing Runnable Interface (Preferred)

```
class MyTask implements Runnable {
    private String taskName;
    MyTask(String name) { taskName = name; }

    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(taskName + " - Step: " + i);
        }
    }
}

public class RunnableDemo {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyTask("Task-A"));
        Thread t2 = new Thread(new MyTask("Task-B"));
        t1.start();
        t2.start();
    }
}
```

9.4 Thread Methods

Method	Description
<code>start()</code>	Starts the thread — calls <code>run()</code> in new thread
<code>run()</code>	Contains the thread's code (override this)
<code>sleep(ms)</code>	Pauses thread for given milliseconds
<code>join()</code>	Waits for the thread to finish before continuing
<code>getName()</code>	Returns thread name
<code>getPriority()</code>	Returns thread priority (1–10)
<code>isAlive()</code>	Returns true if thread is still running
<code>yield()</code>	Temporarily pauses to let other threads run

9.5 Thread Synchronization

When multiple threads access shared data simultaneously, it can cause data corruption. Synchronization ensures only ONE thread accesses a resource at a time.

```
class Counter {
    private int count = 0;

    // synchronized method — only 1 thread at a time
    synchronized void increment() {
        count++;
    }

    int getCount() { return count; }
}
```

```

class MyThread extends Thread {
    Counter c;
    MyThread(Counter c) { this.c = c; }

    public void run() {
        for (int i = 0; i < 1000; i++) {
            c.increment();
        }
    }
}

public class SyncDemo {
    public static void main(String[] args) throws InterruptedException {
        Counter c = new Counter();
        MyThread t1 = new MyThread(c);
        MyThread t2 = new MyThread(c);
        t1.start(); t2.start();
        t1.join(); t2.join();
        System.out.println("Count: " + c.getCount()); // 2000
    }
}

```



Output:

Count: 2000



Golden Rules / Important Notes

- Always implement Runnable (preferred over extending Thread) — keeps inheritance free
- call start(), NOT run() — calling run() directly runs in the SAME thread
- synchronized prevents race condition — but can cause deadlock if overused
- Thread.sleep() throws InterruptedException — must handle it
- join() ensures main thread waits for other threads to finish



Exam-Oriented Questions

- Q: What is multithreading? Explain thread lifecycle with diagram.
- Q: What are the two ways to create a thread? Which is preferred and why?
- Q: What is synchronization? Why is it needed? Demonstrate with program.

UNIT 10 — File Handling

10.1 Introduction

Java provides the `java.io` package to work with files. File handling allows reading from and writing to files on disk — data persists even after program ends.

```
File Handling Classes (java.io package):
File           - Represents file/directory path
FileWriter     - Write characters to file
FileReader     - Read characters from file
BufferedWriter - Buffered writing (efficient)
BufferedReader - Buffered reading (efficient)
FileInputStream - Read bytes from file
FileOutputStream - Write bytes to file
```

10.2 Writing to a File

```
import java.io.*;

public class WriteFile {
    public static void main(String[] args) {
        try {
            // FileWriter: creates file if not exists, overwrites if exists
            FileWriter fw = new FileWriter("notes.txt");
            BufferedWriter bw = new BufferedWriter(fw);

            bw.write("Hello, this is line 1");
            bw.newLine(); // New line
            bw.write("This is line 2");
            bw.newLine();
            bw.write("Java File Handling is easy!");

            bw.close(); // IMPORTANT: always close
            System.out.println("File written successfully!");

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```



Output:

```
File written successfully!
```

(File 'notes.txt' is created with 3 lines of text)

10.3 Reading from a File

```
import java.io.*;

public class ReadFile {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("notes.txt");
            BufferedReader br = new BufferedReader(fr);

            String line;
```

```

        System.out.println("--- File Contents ---");
        while ((line = br.readLine()) != null) {
            System.out.println(line);
        }
        br.close();

    } catch (FileNotFoundException e) {
        System.out.println("File not found!");
    } catch (IOException e) {
        System.out.println("Error reading: " + e.getMessage());
    }
}

```

Output:

```

--- File Contents ---
Hello, this is line 1
This is line 2
Java File Handling is easy!

```

10.4 File Class — Useful Methods

```

import java.io.*;

public class FileInfo {
    public static void main(String[] args) {
        File f = new File("notes.txt");
        System.out.println("Name: " + f.getName());
        System.out.println("Path: " + f.getAbsolutePath());
        System.out.println("Exists: " + f.exists());
        System.out.println("Readable: " + f.canRead());
        System.out.println("Size: " + f.length() + " bytes");
        System.out.println("Is Dir: " + f.isDirectory());
        // f.delete(); // Delete the file
        // f.mkdir(); // Create directory
    }
}

```

10.5 try-with-resources (Java 7+)

Automatically closes resources — no need for manual close(). Cleaner and safer code.

```

import java.io.*;

public class TryWithResources {
    public static void main(String[] args) {
        // Resource automatically closed after try block
        try (BufferedReader br = new BufferedReader(new FileReader("notes.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        // br is automatically closed here
    }
}

```



Golden Rules / Important Notes

- ALWAYS close streams — resource leak causes serious problems
- Use try-with-resources for automatic closing (Java 7+)

- `FileWriter("file.txt", true)` — 'true' means APPEND mode (don't overwrite)
- `FileNotFoundException` is thrown if file doesn't exist while reading
- `BufferedReader/Writer` are more efficient than plain `FileReader/Writer`



Exam-Oriented Questions

- Q: Explain file handling in Java. List important classes used.
 - Q: Write a Java program to write data to a file and then read it back.
 - Q: What is try-with-resources? Why is it preferred?
-

UNIT 11 — Collections Framework

11.1 Introduction

The Java Collections Framework provides ready-made data structures and algorithms. It's in the `java.util` package and is essential for real-world programming.

```
Collections Hierarchy:
Iterable
└─ Collection
    ├── List (ordered, allows duplicates)
    │   ├── ArrayList
    │   ├── LinkedList
    │   └── Vector
    ├── Set (no duplicates)
    │   ├── HashSet
    │   ├── LinkedHashSet
    │   └── TreeSet (sorted)
    └── Queue
        ├── PriorityQueue
        └── LinkedList
Map (key-value pairs — NOT from Collection)
├── HashMap (no order)
├── LinkedHashMap (insertion order)
└── TreeMap (sorted by key)
```

11.2 ArrayList

`ArrayList` is a dynamic array — size grows automatically. Maintains insertion order. Allows duplicates. Best for frequent read operations.

```
import java.util.*;

public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();

        // Adding elements
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");
        list.add("Apple"); // Duplicates allowed

        System.out.println("List: " + list);
        System.out.println("Size: " + list.size());
        System.out.println("Element at 1: " + list.get(1));

        list.remove("Banana"); // Remove by value
        list.remove(0);        // Remove by index

        System.out.println("After removal: " + list);

        // Iterating
        System.out.println("--- Iterator ---");
        for (String fruit : list) {
            System.out.println(fruit);
        }

        Collections.sort(list); // Sort alphabetically
    }
}
```

```

        System.out.println("Sorted: " + list);
    }
}

```

Output:

```

List: [Apple, Banana, Cherry, Apple]
Size: 4
Element at 1: Banana
After removal: [Cherry, Apple]
--- Iterator ---
Cherry
Apple
Sorted: [Apple, Cherry]

```

11.3 LinkedList

LinkedList implements both List and Deque. Efficient for frequent insertions/deletions. Can be used as Stack or Queue.

```

LinkedList<Integer> ll = new LinkedList<>();
ll.add(10); ll.add(20); ll.add(30);
ll.addFirst(5);    // Add at beginning
ll.addLast(40);    // Add at end
System.out.println(ll);           // [5, 10, 20, 30, 40]
System.out.println(ll.getFirst()); // 5
System.out.println(ll.getLast());  // 40
ll.removeFirst();
System.out.println(ll);           // [10, 20, 30, 40]

```

11.4 HashSet

HashSet stores unique elements only. No guaranteed order. Uses hash table internally. $O(1)$ for add, remove, contains.

```

HashSet<String> set = new HashSet<>();
set.add("Java");
set.add("Python");
set.add("Java");    // Duplicate - ignored
set.add("C++");
System.out.println(set); // [Python, Java, C++] (order may vary)
System.out.println(set.contains("Java")); // true
set.remove("C++");
System.out.println(set.size()); // 2

```

11.5 HashMap

HashMap stores data as key-value pairs. Keys must be unique. Values can be duplicated. No guaranteed order.

```

import java.util.*;

public class HashMapDemo {
    public static void main(String[] args) {
        HashMap<String, Integer> map = new HashMap<>();

        // Add key-value pairs
        map.put("Rahul", 85);
        map.put("Priya", 92);
        map.put("Amit", 78);
        map.put("Priya", 95); // Overwrites previous value

        System.out.println("Map: " + map);
    }
}

```

```

        System.out.println("Priya's marks: " + map.get("Priya"));
        System.out.println("Has Rahul? " + map.containsKey("Rahul"));

        // Iterate
        for (Map.Entry<String, Integer> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " → " + entry.getValue());
        }

        map.remove("Amit");
        System.out.println("Size: " + map.size());
    }
}

```

Output:

```

Map: {Rahul=85, Priya=95, Amit=78}
Priya's marks: 95
Has Rahul? true
Rahul → 85
Priya → 95
Amit → 78
Size: 2

```

11.6 Collections Comparison Table

Collection	Order	Duplicates	Null	Thread Safe	Best For
<code>ArrayList</code>	Insertion	Yes	Yes	No	Frequent reads
<code>LinkedList</code>	Insertion	Yes	Yes	No	Frequent insert/delete
<code>HashSet</code>	No order	No	1	No	Unique elements
<code>TreeSet</code>	Sorted	No	No	No	Sorted unique
<code>HashMap</code>	No order	Keys: No	1 key	No	Key-value fast lookup
<code>TreeMap</code>	Sorted key	Keys: No	No	No	Sorted key-value
<code>Vector</code>	Insertion	Yes	Yes	Yes	Thread-safe list

Golden Rules / Important Notes

- `ArrayList`: best for READ. `LinkedList`: best for INSERT/DELETE
- Set does not allow duplicates; List allows duplicates
- `HashMap` key must be unique; `map.put(key, value)` overwrites existing value
- Always use generics: `ArrayList<String>` — type safety at compile time
- `Collections.sort()` — sorts List; `TreeSet/TreeMap` auto-sort

Exam-Oriented Questions

- Q: Explain Java Collections Framework. Draw the hierarchy.
- Q: Differentiate between `ArrayList` and `LinkedList`.
- Q: Write a program using `HashMap` to store student names and marks.
- Q: What is the difference between List, Set, and Map?

UNIT 12 — GUI Programming (AWT & Swing)

12.1 Introduction to GUI

Java provides two main frameworks for creating Graphical User Interfaces (GUI): AWT (Abstract Window Toolkit) — older, platform-dependent, and Swing — modern, platform-independent, more components.

Feature	AWT	Swing
Package	java.awt	javax.swing
Platform	Platform-dependent (native)	Platform-independent
Components	Heavyweight	Lightweight
Look & Feel	Native OS look	Consistent across platforms
Components	Button, TextField	JButton, JTextField
Performance	Faster	Slightly slower

12.2 Swing — Basic Components

► JFrame — Main Window

```
import javax.swing.*;
import java.awt.*;

public class SimpleWindow extends JFrame {
    SimpleWindow() {
        setTitle("My First Java GUI");    // Window title
        setSize(400, 300);                // Width x Height
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);      // Center on screen
        setVisible(true);                 // Show window
    }

    public static void main(String[] args) {
        new SimpleWindow();
    }
}
```

► Complete Swing Example with Components

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class SwingDemo extends JFrame implements ActionListener {
    JLabel lblName;
    JTextField txtName;
    JButton btnGreet;
    JLabel lblResult;

    SwingDemo() {
        setTitle("Greeting App");
        setSize(350, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

```

        setLayout(new FlowLayout()); // Layout manager

        // Create components
        lblName = new JLabel("Enter Name:");
        txtName = new JTextField(15);
        btnGreet = new JButton("Greet");
        lblResult = new JLabel("");

        // Add action listener to button
        btnGreet.addActionListener(this);

        // Add components to frame
        add(lblName); add(txtName);
        add(btnGreet); add(lblResult);

        setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        // Called when button is clicked
        String name = txtName.getText();
        lblResult.setText("Hello, " + name + "!");
    }

    public static void main(String[] args) {
        new SwingDemo();
    }
}

```

12.3 Common Swing Components

Component	Purpose	Example
JFrame	Main window	new JFrame("Title")
JPanel	Container for components	new JPanel()
JLabel	Display text/image	new JLabel("Name:")
JTextField	Single-line text input	new JTextField(20)
TextArea	Multi-line text input	new JTextArea(5, 20)
JButton	Clickable button	new JButton("Click Me")
JCheckBox	Toggle selection	new JCheckBox("Agree")
JRadioButton	Single selection	new JRadioButton("Male")
JComboBox	Dropdown list	new JComboBox<>(options)
JList	Scrollable list	new JList<>(data)
JMenuBar	Menu bar at top	new JMenuBar()
JMenu	Dropdown menu	new JMenu("File")
JMenuItem	Menu option	new JMenuItem("Open")

12.4 Layout Managers

- — Left to right, top to bottom (default for JPanel)
- — North, South, East, West, Center (default for JFrame)

- — Equal-sized grid: `new GridLayout(rows, cols)`
- — Vertical or horizontal stack
- — Absolute positioning with `setBounds(x, y, w, h)`

```
// BorderLayout example
setLayout(new BorderLayout());
add(new JButton("North"), BorderLayout.NORTH);
add(new JButton("South"), BorderLayout.SOUTH);
add(new JButton("East"), BorderLayout.EAST);
add(new JButton("West"), BorderLayout.WEST);
add(new JButton("Center"), BorderLayout.CENTER);
```

⚡ Golden Rules / Important Notes

- `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` — required or window won't close
- `setVisible(true)` — must be called LAST after adding all components
- `ActionListener` — interface for button events; override `actionPerformed()`
- `EventQueue.invokeLater()` — create GUI on Event Dispatch Thread for thread safety

📝 Exam-Oriented Questions

- Q: What is the difference between AWT and Swing?
 - Q: Write a Swing program with `JFrame`, `JLabel`, `TextField`, and `JButton`.
 - Q: Explain any 4 layout managers in Java Swing.
 - Q: What is an event listener? Explain `ActionListener` with example.
-

UNIT 13 — JDBC (Java Database Connectivity)

13.1 Introduction to JDBC

JDBC is a Java API that allows Java programs to connect and interact with databases. It provides a standard interface to execute SQL queries, update records, and retrieve results.

Key Points

- JDBC is in the `java.sql` package
- Works with MySQL, Oracle, SQLite, PostgreSQL, etc.
- JDBC Driver: acts as a bridge between Java and Database
- 4 types of JDBC drivers; Type 4 (Thin driver) is most common

```
JDBC Architecture:
Java Application
  ↓
JDBC API (java.sql.*)
  ↓
JDBC Driver Manager
  ↓
JDBC Driver (e.g., MySQL Connector/J)
  ↓
Database (MySQL, Oracle, etc.)
```

13.2 JDBC Steps (5 Steps)

Always Follow These 5 Steps

- Step 1: Load the Driver Class
- Step 2: Establish Connection
- Step 3: Create Statement
- Step 4: Execute Query
- Step 5: Close Connection

13.3 Important JDBC Interfaces

Interface/Class	Purpose
<code>DriverManager</code>	Manages a list of database drivers; <code>getConnection()</code>
<code>Connection</code>	Represents a connection to the database
<code>Statement</code>	Executes simple SQL queries
<code>PreparedStatement</code>	Executes pre-compiled SQL (prevents SQL injection)
<code>ResultSet</code>	Holds data returned by a <code>SELECT</code> query
<code>CallableStatement</code>	Executes stored procedures

13.4 Complete JDBC Example (MySQL)

This example connects to MySQL, inserts data, and retrieves it. Assumes MySQL is running with a 'college' database.

Database Setup (SQL):

```
-- Run these SQL commands in MySQL first:
CREATE DATABASE college;
```

```
USE college;
CREATE TABLE student (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50),
    marks INT
);
```

Java JDBC Program:

```
import java.sql.*;

public class JdbcDemo {
    // Connection details
    static final String URL = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root";
    static final String PASS = "your_password";

    public static void main(String[] args) {
        Connection con = null;
        Statement stmt = null;

        try {
            // Step 1: Load Driver (Java 6+ auto-loads, but good to know)
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Step 2: Establish Connection
            con = DriverManager.getConnection(URL, USER, PASS);
            System.out.println("Connected to database!");

            // Step 3: Create Statement
            stmt = con.createStatement();

            // Step 4a: INSERT data
            stmt.executeUpdate(
                "INSERT INTO student(name, marks) VALUES('Rahul', 85)"
            );
            stmt.executeUpdate(
                "INSERT INTO student(name, marks) VALUES('Priya', 92)"
            );
            System.out.println("Records inserted!");

            // Step 4b: SELECT data
            ResultSet rs = stmt.executeQuery("SELECT * FROM student");

            System.out.println("\nID | Name | Marks");
            System.out.println("----+-----+-----");
            while (rs.next()) {
                System.out.printf("%2d | %-5s | %d\n",
                    rs.getInt("id"),
                    rs.getString("name"),
                    rs.getInt("marks")
                );
            }
            rs.close();

        } catch (ClassNotFoundException e) {
            System.out.println("Driver not found: " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("SQL Error: " + e.getMessage());
        } finally {
            // Step 5: Close Connection
            try {
                if (stmt != null) stmt.close();
                if (con != null) con.close();
            } catch (SQLException e) {
                // Ignored
            }
        }
    }
}
```

```

        System.out.println("Connection closed.");
    } catch (SQLException e) { e.printStackTrace(); }
}
}
}

```

Output:

```

Connected to database!
Records inserted!

```

```

ID | Name | Marks
---+-----+-----
1  | Rahul | 85
2  | Priya | 92
Connection closed.

```

13.5 PreparedStatement — Safer Queries

PreparedStatement is preferred over Statement because it prevents SQL injection and is faster for repeated queries.

```

// PreparedStatement with parameters
String sql = "INSERT INTO student(name, marks) VALUES(?, ?)";
PreparedStatement pstmt = con.prepareStatement(sql);

// Set values (1-indexed)
pstmt.setString(1, "Anjali");
pstmt.setInt(2, 88);
pstmt.executeUpdate();

// Reuse for another record
pstmt.setString(1, "Ravi");
pstmt.setInt(2, 76);
pstmt.executeUpdate();

pstmt.close();

```

13.6 CRUD Operations Summary

Operation	SQL	JDBC Method
CREATE	INSERT INTO ...	stmt.executeUpdate(sql)
READ	SELECT * FROM ...	stmt.executeQuery(sql) → ResultSet
UPDATE	UPDATE ... SET ...	stmt.executeUpdate(sql)
DELETE	DELETE FROM ...	stmt.executeUpdate(sql)

Golden Rules / Important Notes

- Always close Connection, Statement, ResultSet in finally block (or use try-with-resources)
- Use PreparedStatement instead of Statement — prevents SQL injection
- executeQuery() → returns ResultSet (for SELECT)
- executeUpdate() → returns int (rows affected) (for INSERT/UPDATE/DELETE)
- Add mysql-connector-java.jar to classpath before running JDBC programs

Exam-Oriented Questions

- Q: What is JDBC? Explain the steps to connect Java with MySQL.
- Q: What is PreparedStatement? How is it different from Statement?
- Q: Write a complete JDBC program to display all records from a student table.
- Q: Explain CRUD operations in JDBC with SQL and Java code.



Quick Revision — Exam Cheat Sheet

Key Definitions at a Glance

Topic	One-Line Definition
Java	Platform-independent, OOP language by James Gosling (1995)
JVM	Executes bytecode; platform-specific; provides runtime environment
JDK	JRE + dev tools (javac, java); used by developers
OOP	Programming paradigm using objects: Encapsulation, Inheritance, Polymorphism, Abstraction
Class	Blueprint/template for objects
Object	Instance of a class; created using 'new' keyword
Constructor	Special method to initialize objects; same name as class, no return type
Inheritance	'extends' keyword; child gets parent's methods & fields
Overloading	Same method name, different parameters in SAME class
Overriding	Child redefines parent's method with SAME signature
Interface	100% abstract; 'implements' keyword; supports multiple inheritance
Package	Namespace to group related classes; 'package' keyword
Exception	Runtime error; handled using try-catch-finally
Thread	Lightweight process; 'extends Thread' or 'implements Runnable'
synchronized	Allows only 1 thread to access shared resource at a time
Collection	Framework for data structures: List, Set, Map
ArrayList	Dynamic array; ordered; duplicates allowed
HashMap	Key-value pairs; no order; keys unique
JDBC	API to connect Java with databases; 5 steps
Swing	Platform-independent GUI toolkit; javax.swing package

Most Important Programs — Exam Ready



Must-Practice Programs

- Hello World (class, main, print)
- Fibonacci series using loop
- Factorial using recursion
- Array sort and search
- String reverse and palindrome check
- Class with constructor + methods (OOP)
- Single/Multilevel inheritance
- Interface implementation
- try-catch-finally exception handling
- Custom exception class
- Thread using Runnable interface
- Read/Write file using BufferedReader/Writer

- ArrayList CRUD operations
- HashMap student records
- JDBC connect + INSERT + SELECT
- JFrame with JButton and ActionListener



Best of Luck for Your Exams!

Study smart, practice daily, and you'll ace it! 🎯